Methodology for Addition Causal Abstraction Experiments

Repo draft

March 16, 2026

Abstract

This document describes the end-to-end methodology implemented in the addition experiment codebase. The setup trains a single supervised neural network for two-digit addition, constructs counterfactual abstract interventions from a symbolic structural causal model, and then compares optimal transport (OT), Gromov–Wasserstein (GW), fused Gromov–Wasserstein (FGW), and distributed alignment search (DAS) as mechanisms for mapping abstract variables to neural network sites. The notation is standardized around abstract variables Z_m for $m = 1, \ldots, M$ and neural sites S_n for $n = 1, \ldots, N$.

1 Problem Setup

The input is a pair of base-10 two-digit integers. Each input digit is encoded as a one-hot vector in $\{0,1\}^{10}$, and the four digit vectors are concatenated, giving a neural input space

$$x \in \{0,1\}^{40}.$$

The structural causal model (SCM) contains the intermediate addition variables

$$Z_1 = S_1, \quad Z_2 = C_1, \quad Z_3 = S_2, \quad Z_4 = C_2,$$

where S_1 and C_1 are the sum and carry at the least significant digit, and S_2 and C_2 are the corresponding variables at the next digit after propagating C_1. By default the implementation considers all four abstract variables, so $M = 4$.

The final output is the full sum

$$Y = 100C_2 + 10S_2 + S_1 \in \{0, \ldots, 199\},$$

which is treated as a 200-class classification problem.

2 Neural Model

The neural model is a four-hidden-layer ReLU multilayer perceptron with hidden width 192 at every hidden layer:

$$40 \to 192 \to 192 \to 192 \to 192 \to 200.$$

Let $h^{(\ell)}(x) \in \mathbb{R}^{192}$ denote the hidden activation vector at layer $\ell \in \{1,2,3,4\}$. The output layer is followed by a softmax, and we absorb that softmax into the neural network map

$$f_\theta(x) \in \Delta^{200},$$

where Δ^{200} is the probability simplex over the 200 output classes. The model is trained with multiclass cross-entropy loss.

The default factual supervised protocol uses:

- 30,000 training examples per seed,
- 4,000 validation examples per seed,
- a fixed random seed that controls both factual and counterfactual data generation.

3 Counterfactual Pair Protocol

After training the neural network, all abstraction methods operate on the same trained MLP and the same abstract variables $\{Z_m\}_{m=1}^{M}$. Counterfactual fitting is driven by a pair bank

$$\mathcal{T} = \{(x_t^b, x_t^s)\}_{t=1}^{T},$$

where x_t^b is the base input and x_t^s is the source input. Let the abstract model be denoted by g. Then the factual abstract output is

$$y_t^{abs} = g(x_t^b),$$

and the abstract counterfactual output obtained by swapping variable Z_m from the source world into the base world is

$$y_{m,t}^{abs\text{-}swap} = g\big(x_t^b, \mathrm{do}(Z_m \leftarrow Z_m(x_t^s))\big).$$

Let the neural network output on the factual base input be

$$y_t^{nn} = f_\theta(x_t^b),$$

and, for any replacement vector $\widetilde{h} \in \mathbb{R}^{d_\ell}$ at hidden layer ℓ, write

$$f_\theta\big(x_t^b, \mathrm{do}(h^{(\ell)}(x_t^b) \leftarrow \widetilde{h})\big)$$

for the output of the forward pass on x_t^b in which the hidden activation at layer ℓ is overwritten by $\widetilde{h}$.

The default pair splits are disjoint:

- train split $\mathcal{T}_{\text{train}}$ with 48 base/source pairs,
- calibration split $\mathcal{T}_{\text{cal}}$ with 48 base/source pairs,
- test split: 96 base/source pairs.

These splits are used uniformly across methods:

- GW, OT, and FGW fit alignment statistics on $\mathcal{T}_{\text{train}}$,
- GW, OT, and FGW then select per-variable top-K_m truncation levels and one shared λ on $\mathcal{T}_{\text{cal}}$,
- DAS trains candidate rotations on $\mathcal{T}_{\text{train}}$,
- DAS selects (layer, k) on $\mathcal{T}_{\text{cal}}$,
- final exact-match and digit-overlap metrics are reported on the shared test split.

At a high level, every method defines, for each abstract variable Z_m, an abstract-variable-level neural counterfactual output $y_{m,t}^{nn\text{-}swap}$ intended to approximate $y_{m,t}^{abs\text{-}swap}$. The methods differ only in how this neural intervention is parameterized, learned, or optimized.

4 Canonical-Basis Baseline

For the canonical-basis baseline, a candidate neural site S_n is a coordinate subspace in a single hidden layer. Concretely, we write

$$S_n = (\ell, I_n),$$

where $\ell \in \{1, \ldots, L\}$ is a hidden-layer index and $I_n \subseteq \{1, \ldots, d_\ell\}$ is a subset of canonical coordinates in that layer. Single-neuron swaps are the special case $|I_n| = 1$, but the canonical-basis baseline is not restricted to that case.

For a candidate $S_n = (\ell, I_n)$, the intervened hidden state is

$$h^{(\ell)}(x_t^b; S_n \leftarrow x_t^s) = h^{(\ell)}(x_t^b) + \sum_{i \in I_n} \Big(\big[h^{(\ell)}(x_t^s)\big]_i - \big[h^{(\ell)}(x_t^b)\big]_i \Big) e_i,$$

with all other layers left unchanged. The resulting neural output is

$$y_{n,t}^{\textit{nn-swap}} = f_\theta\Big(x_t^b, \mathrm{do}\big(h^{(\ell)}(x_t^b) \leftarrow h^{(\ell)}(x_t^b; S_n \leftarrow x_t^s)\big)\Big).$$

For each abstract variable Z_m, we sweep over a prescribed family of canonical subspaces and retain the best-performing candidate. This gives a hard-swap baseline that uses the canonical basis directly, without learned rotations or transport weights.

5 DAS

For DAS, a candidate site is parameterized by a pair (ℓ, k), where $\ell \in \{1, \ldots, L\}$ is a hidden-layer index and $k \in \{1, \ldots, d_\ell\}$ is a subspace dimension. Here d_ℓ denotes the width of hidden layer ℓ; in the default network, $L = 4$ and $d_1 = d_2 = d_3 = d_4 = 192$.

The location of the k coordinates does not matter in the canonical basis, because DAS learns a rotation matrix $R_\ell \in \mathbb{R}^{d_\ell \times d_\ell}$ for each layer. After rotation, the first k coordinates define the intervened subspace. Thus DAS is choosing a layer and a subspace dimension, not a fixed canonical coordinate subset.

For a candidate (ℓ, k), write the base and source hidden states at layer ℓ as $h^{(\ell)}(x_t^b)$ and $h^{(\ell)}(x_t^s)$. Define the corresponding rotated activations

$$\widetilde{h}^{(\ell)}(x_t^b) = R_\ell h^{(\ell)}(x_t^b), \qquad \widetilde{h}^{(\ell)}(x_t^s) = R_\ell h^{(\ell)}(x_t^s),$$

and swap the first k rotated coordinates:

$$\widetilde{h}^{(\ell)}(x_t^b; Z_m \leftarrow x_t^s)(1{:}k) = \widetilde{h}^{(\ell)}(x_t^s)(1{:}k), \qquad \widetilde{h}^{(\ell)}(x_t^b; Z_m \leftarrow x_t^s)(k{+}1{:}d_\ell) = \widetilde{h}^{(\ell)}(x_t^b)(k{+}1{:}d_\ell).$$

Then rotate back:

$$h^{(\ell)}(x_t^b; Z_m \leftarrow x_t^s) = R_\ell^{-1} \widetilde{h}^{(\ell)}(x_t^b; Z_m \leftarrow x_t^s).$$

This produces a DAS neural counterfactual output

$$y_{m,t}^{\textit{nn-swap}}(\ell, k) = f_\theta\Big(x_t^b, \mathrm{do}\big(h^{(\ell)}(x_t^b) \leftarrow h^{(\ell)}(x_t^b; Z_m \leftarrow x_t^s)\big)\Big),$$

where the intervened hidden state on the right-hand side is the rotated-subspace swap defined above for abstract variable Z_m at layer ℓ and dimension k. The rotation parameters are trained on the train split with cross-entropy loss against the abstract counterfactual targets $y_{m,t}^{\textit{abs-swap}}$, and the best (ℓ, k) is selected on the calibration split by interchange intervention accuracy.

6 Transport Methods

For OT, GW, and FGW, we instantiate the neural site family at resolution 1, so each transport site is a single canonical neuron. Thus each site is a pair (ℓ, i) with neuron index $i \in \{1, \ldots, d_\ell\}$ in hidden layer ℓ. The total number of transport sites is therefore

$$N = \sum_{\ell=1}^{L} d_\ell.$$

We write $S_n \equiv (\ell, i)$ when transport-site index n corresponds to neuron i in layer ℓ.

6.1 Effect Signatures

For abstract variable Z_m, the abstract effect on example t is

$$\delta_{m,t}^{abs} = \text{onehot}(y_{m,t}^{abs\text{-}swap}) - \text{onehot}(y_t^{abs}),$$

which lies in $\mathbb{R}^{200}$. Concatenating across the T fit pairs gives the abstract signature

$$u_m^{abs} = \left[\delta_{m,1}^{abs}; \ldots; \delta_{m,T}^{abs}\right] \in \mathbb{R}^{200T}.$$

For canonical site S_n, the neural effect on example t is

$$\delta_{n,t}^{nn} = y_{n,t}^{nn\text{-}swap} - y_t^{nn},$$

which also lies in $\mathbb{R}^{200}$. Concatenating these over the T fit pairs gives the neural signature

$$v_n^{nn} = \left[\delta_{n,1}^{nn}; \ldots; \delta_{n,T}^{nn}\right] \in \mathbb{R}^{200T}.$$

6.2 Metric-Measure Spaces and Alignment Objectives

Define the abstract metric-measure space

$$\mathcal{X}^{abs} = \left(\{u_m^{abs}\}_{m=1}^{M}, d^{abs}, p\right),$$

where

$$d^{abs}(u_m^{abs}, u_{m'}^{abs}) = \|u_m^{abs} - u_{m'}^{abs}\|_2, \qquad p_m = \frac{1}{M}.$$

Similarly, define the neural metric-measure space

$$\mathcal{X}^{nn} = \left(\{v_n^{nn}\}_{n=1}^{N}, d^{nn}, q\right),$$

where

$$d^{nn}(v_n^{nn}, v_{n'}^{nn}) = \|v_n^{nn} - v_{n'}^{nn}\|_2, \qquad q_n = \frac{1}{N}.$$

Direct OT uses the cross-space cost matrix

$$\mathbf{M}_{m,n} = \|u_m^{abs} - v_n^{nn}\|_2$$

and solves

$$\min_{\mathbf{T} \in \Pi(p,q)} \langle \mathbf{T}, \mathbf{M} \rangle.$$

With entropic regularization, this becomes

$$\min_{\mathbf{T}\in\Pi(p,q)} \langle \mathbf{T}, \mathbf{M}\rangle + \varepsilon \sum_{m,n} \mathbf{T}_{m,n}(\log \mathbf{T}_{m,n} - 1).$$

GW instead compares the internal geometries of the two spaces. Define

$$\mathbf{C}^{Z}_{m,m'} = d^{abs}(u^{abs}_m, u^{abs}_{m'}) = \|u^{abs}_m - u^{abs}_{m'}\|_2,$$
$$\mathbf{C}^{S}_{n,n'} = d^{nn}(v^{nn}_n, v^{nn}_{n'}) = \|v^{nn}_n - v^{nn}_{n'}\|_2.$$

GW then solves

$$\min_{\mathbf{T}\in\Pi(p,q)} \sum_{m,m',n,n'} L\big(\mathbf{C}^{Z}_{m,m'}, \mathbf{C}^{S}_{n,n'}\big)\ \mathbf{T}_{m,n}\mathbf{T}_{m',n'},$$

with $L(a,b) = (a-b)^2$.

FGW interpolates between the direct OT cost and the GW structural cost:

$$\min_{\mathbf{T}\in\Pi(p,q)} (1-\alpha)\langle \mathbf{T}, \mathbf{M}\rangle + \alpha \sum_{m,m',n,n'} L\big(\mathbf{C}^{Z}_{m,m'}, \mathbf{C}^{S}_{n,n'}\big)\ \mathbf{T}_{m,n}\mathbf{T}_{m',n'}.$$

Implementation note: the current code uses POT's `ot.sinkhorn`, `ot.gromov.entropic_gromov_wasserstein`, and `ot.gromov.BAPG_fused_gromov_wasserstein` for OT, GW, and FGW respectively.

6.3 Transport-Induced Neural Swap

For OT, GW, and FGW, the learned transport plan is converted into a soft intervention after a calibration-stage truncation-and-scaling sweep. For abstract variable Z_m, let

$$\bar{\mathbf{T}}_{m,n} = \frac{\mathbf{T}_{m,n}}{\sum_{n'} \mathbf{T}_{m,n'}}$$

denote the row-normalized transport weights over canonical neural sites.

For a chosen top-K_m value for row m, let TopK_m be the set of the K_m highest-mass sites in row m of $\bar{\mathbf{T}}$. The retained mass is then renormalized within that row:

$$\widetilde{\mathbf{T}}_{m,n} = \frac{\mathbf{1}[n \in \mathrm{TopK}_m]\bar{\mathbf{T}}_{m,n}}{\sum_{n'} \mathbf{1}[n' \in \mathrm{TopK}_m]\bar{\mathbf{T}}_{m,n'}}.$$

Let $\lambda > 0$ be a shared intervention-strength parameter used for every abstract variable.

Using the identification $S_n \equiv (\ell, i)$, the transport-based intervention at layer ℓ is

$$h^{(\ell)}(x^b_t; Z_m \leftarrow x^s_t) = h^{(\ell)}(x^b_t) + \sum_{i=1}^{d_\ell} \lambda\, \widetilde{\mathbf{T}}_{m,(\ell,i)} \left(\big[h^{(\ell)}(x^s_t)\big]_i - \big[h^{(\ell)}(x^b_t)\big]_i \right) e_i.$$

This is applied sequentially across all hidden layers, yielding the abstract-variable-level neural counterfactual output

$$y^{nn\text{-}swap}_{m,t} = f_\theta\Big(x^b_t, \mathrm{do}\big(h^{(1)}(x^b_t) \leftarrow h^{(1)}(x^b_t; Z_m \leftarrow x^s_t), \ldots, h^{(L)}(x^b_t) \leftarrow h^{(L)}(x^b_t; Z_m \leftarrow x^s_t)\big)\Big),$$

where each layerwise intervened hidden state is the soft transport swap defined above for abstract variable Z_m. For each candidate shared λ, the code selects the best top-K_m separately for each abstract variable by calibration exact-match accuracy, with shared-digit score used as a tie-breaker. It then chooses the shared λ by macro calibration exact-match accuracy averaged over abstract variables, again using macro shared-digit score as a tie-breaker. Thus, the canonical-basis baseline, OT, GW, and FGW all modify the same object: they differ only in how the site weights are specified. The baseline uses a single hard site swap, while OT, GW, and FGW learn soft site weights through the transport plan and then calibrate how much of that mass to apply.

7 Evaluation Metrics

Two metrics are reported on the shared test split.

7.1 Exact-match accuracy

Let $\widehat{y}_{m,t}^{\textit{nn-swap}}$ be the predicted counterfactual class for abstract variable Z_m on pair t, obtained by taking the argmax of the intervened neural output. Exact-match accuracy is

$$\mathrm{Acc}_{\mathrm{exact}}(Z_m) = \frac{1}{N_{\mathrm{test}}} \sum_{t=1}^{N_{\mathrm{test}}} \mathbf{1}\left[\widehat{y}_{m,t}^{\textit{nn-swap}} = y_{m,t}^{\textit{abs-swap}}\right].$$

7.2 Digit Accuracy

Each output class is represented as a zero-padded three-digit tuple

$$(C_2, S_2, S_1).$$

Let $d_p(y)$ denote the pth digit of this tuple, for $p \in \{1, 2, 3\}$. The per-example digit accuracy is

$$\mathrm{Acc}_{\mathrm{digit}}(\widehat{y}, y) = \frac{1}{3} \sum_{p=1}^{3} \mathbf{1}[d_p(\widehat{y}) = d_p(y)].$$

The reported digit score for abstract variable Z_m is then

$$\mathrm{Acc}_{\mathrm{digit}}(Z_m) = \frac{1}{N_{\mathrm{test}}} \sum_{t=1}^{N_{\mathrm{test}}} \mathrm{Acc}_{\mathrm{digit}}\left(\widehat{y}_{m,t}^{\textit{nn-swap}}, y_{m,t}^{\textit{abs-swap}}\right).$$

Equivalently, the code can report the mean number of shared digits, which is just $3 \cdot \mathrm{Acc}_{\mathrm{digit}}(Z_m)$.

8 Default Experimental Configuration

The codebase defaults are therefore:

- input dimension 40,
- output dimension 200,
- four hidden layers with width 192 and ReLU activations,
- factual train/validation sizes 30,000/4,000,
- counterfactual train/calibration/test sizes 48/48/96,
- default abstract variables $\{S_1, C_1, S_2, C_2\}$,
- default OT/GW/FGW canonical sites use resolution 1, so $N = \sum_{\ell=1}^{L} d_\ell$,
- default OT/GW/FGW calibration sweep uses $\lambda \in \{0.5, 0.75, 1.0, 1.25, 1.5\}$ and a top-K grid over powers of two up to the full site count, applied separately to each abstract-variable row,
- default DAS sweep over all subspace sizes $k = 1, \ldots, 192$ for every layer.

9 Experimental Results

The repository's main comparison script writes JSON output to `results/<timestamp>/addition_compare_result` and plot images directly into `results/<timestamp>/`. To compile this draft against a real run, update \`PlotRoot` near the top of the file to the desired timestamped results directory. These figures are intended as a paper scaffold: they demonstrate the reporting format and can be replaced by larger multi-seed plots later without changing the LaTeX structure.

Missing plot: `../results/REPLACE`$_W ITH_T IMESTAMP/exact_accuracy.png$

Figure 1: Per-variable exact-match accuracy for GW, OT, FGW, and DAS on the shared counterfactual test split.

Missing plot: `../results/REPLACE`$_W ITH_T IMESTAMP/shared_digits.png$

Figure 2: Per-variable mean digit-overlap score, computed on zero-padded output triples (C_2, S_2, S_1).

Missing plot: `../results/REPLACE`$_W ITH_T IMESTAMP/macro_summary.png$

Figure 3: Macro summary aggregated over the default abstract-variable set.

10 How to Regenerate the Figures

The main experiment scripts are now meant to be edited directly rather than configured via command-line flags. In practice, the workflow is:

1. edit the config block near the top of `addition_train.py`,
2. run `python addition_train.py`,
3. edit the config block near the top of `addition_compare.py`,
4. run `python addition_compare.py`.

The current default script settings are equivalent to:

```
addition_train.py:
  CHECKPOINT_PATH = "models/addition_mlp.pt"
  OUTPUT_PATH = "results/<timestamp>/addition_train_results.json"
  TRAIN_SIZE = 30000
  VALIDATION_SIZE = 4000
  HIDDEN_DIMS = (192, 192, 192, 192)
  EPOCHS = 100

addition_compare.py:
  CHECKPOINT_PATH = "models/addition_mlp.pt"
  OUTPUT_PATH = "results/<timestamp>/addition_compare_results.json"
  METHODS = ("gw", "ot", "fgw", "das")
```

```
TRAIN_PAIR_SIZE = 48
CALIBRATION_PAIR_SIZE = 48
TEST_PAIR_SIZE = 96
RESOLUTION = 1
OT_TOP_K_VALUES = (1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 768)
OT_LAMBDAS = (0.5, 0.75, 1.0, 1.25, 1.5)
DAS_SUBSPACE_DIMS = (1, 16, 32, 48, 64, 80, 96, 112, 128, 144, 160, 176, 192)
DAS_LAYERS = (0, 1, 2, 3)
DAS_EPOCHS = 5
DAS_LAYERS = None
```